

NAME:AYUSH

SEC:D

SRN:PES1UG24AM909

Architectural Style Selection & Justification

Problem Statement #36 — Restaurant Table Booking & Pre-Ordering App

Architecture Selection: We chose a Microservices Architecture (with a lightweight Reservation Manager acting as an orchestrating service) for the Restaurant Table Booking & Pre-Ordering App.

Reason 1: Independent Scaling for Bursty, Uneven Load

The system's components experience very different load patterns. The Floor Plan Manager and Real-Time Sync Service must push table-status updates to every manager terminal and diner device within 500 ms (NFR-001), which demands high-frequency, low-latency scaling especially during peak dinner hours. In contrast, the Kitchen Timeline Calculator and Payment Gateway are invoked only once per booking. A microservices style lets each capability (floor status, booking, menu/pre-order, kitchen ETA, payment, security) be deployed and scaled independently, so the team can, for example, run multiple replicas of the Real-Time Sync Service without over-provisioning the Payment Gateway.

Reason 2: Fault Isolation Protects the Core Booking Flow

FR-005 depends on a third-party payment gateway, which is the most likely component to experience transient outages or slowdowns. In a microservices design, a failure in the Payment Gateway or an external Kitchen Display integration is isolated to that service and communicated through well-defined interfaces (IPaymentService, IPrepTimeline), rather than crashing the entire monolith. This allows the Reservation Manager to degrade gracefully (e.g., queue the booking and retry payment) instead of blocking table selection and menu browsing for all diners.

Security Advantage

Isolating payment and PII handling inside a dedicated Security & Compliance Service (NFR-002) narrows the PCI-DSS compliance boundary to a single, independently hardened service that enforces AES-256 encryption at rest and TLS 1.3 in transit. Other services (Floor Plan Manager, Menu & Pre-Order) never touch raw card data directly — they only call the ISecureData / IPaymentService interfaces — which reduces the attack surface and simplifies auditing compared to a single monolithic codebase where sensitive and non-sensitive logic are intermixed.

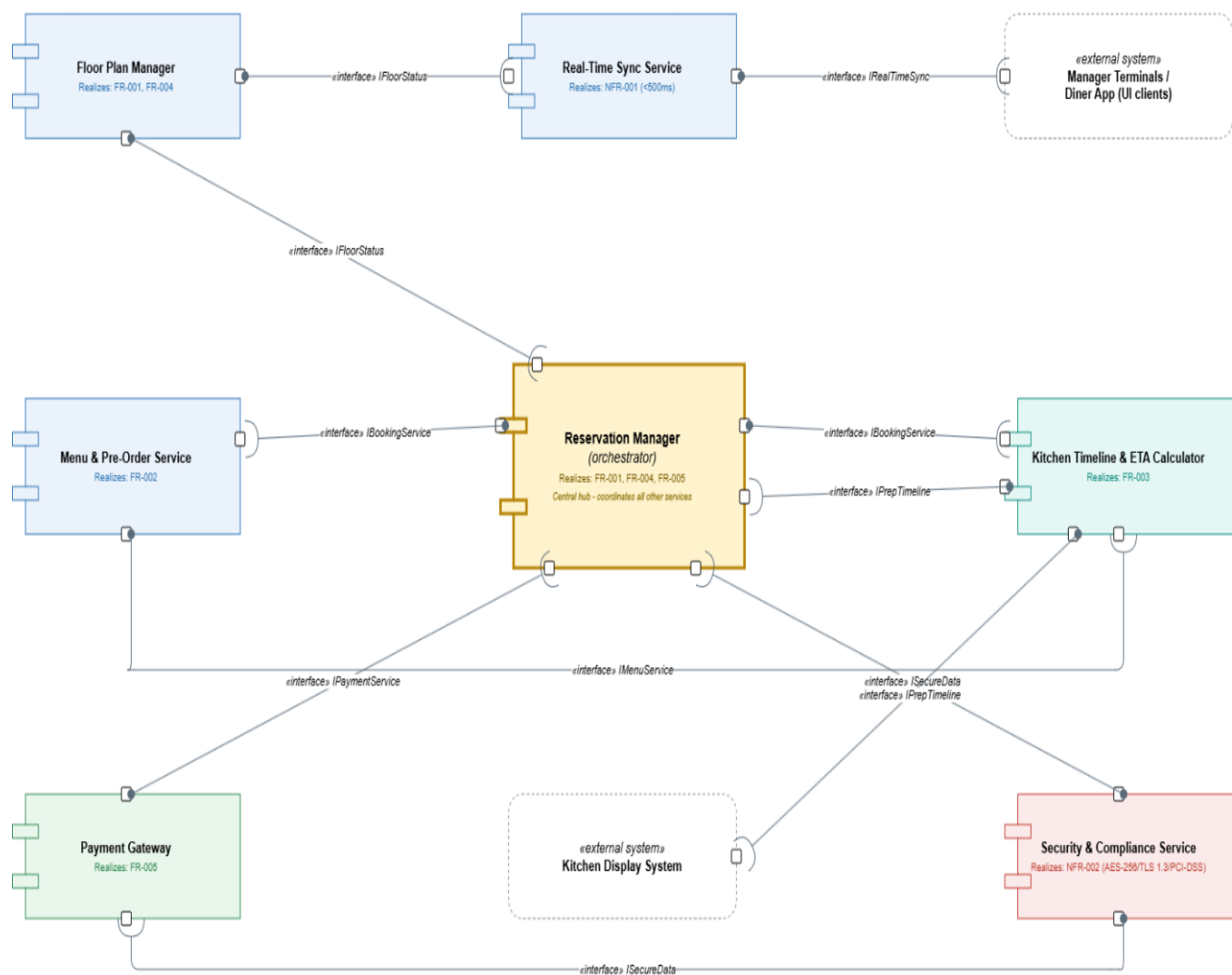
Performance Benefit

Because the Real-Time Sync Service is a separate, independently scalable component, it can be optimized purely for fast, event-driven fan-out (e.g., WebSocket/pub-sub) to meet the sub-500 ms floor-status synchronization requirement in NFR-001, without being slowed down by heavier, less time-critical operations such as kitchen ETA calculation or payment settlement running in the same process.

Component-to-Requirement Traceability

Component	Realizes	Key Interfaces
Floor Plan Manager	FR-001, FR-004	IFloorStatus (provided)
Reservation Manager (orchestrator)	FR-001, FR-004, FR-005	IBookingService (provided); IFloorStatus, IPaymentService, ISecureData, IPrepTimeline (required)
Menu & Pre-Order Service	FR-002	IMenuService (provided); IBookingService (required)
Kitchen Timeline & ETA Calculator	FR-003	IPrepTimeline (provided); IBookingService, IMenuService (required)
Payment Gateway	FR-005	IPaymentService (provided); ISecureData (required)
Real-Time Sync Service	NFR-001 (<500ms sync)	IRealTimeSync (provided); IFloorStatus (required)
Security & Compliance Service	NFR-002 (AES-256 / TLS 1.3 / PCI-DSS)	ISecureData (provided)

UML Component Diagram - Restaurant Table Booking & Pre-Ordering App (PS #36) - with Ports



Legend: ● Provided interface (ball) | ● Required interface (socket) | □ Port (interaction point on boundary) | Amber box = orchestrator | Dashed box = external system
Covers FR-001 to FR-005 and NFR-001, NFR-002 from Lab 1 (Problem Statement #36) - 7 components, 7 interfaces, 22 ports, 11 connectors.